functional programming principles. It applies a function to each element of an array and returns a new array, leaving the original array untouched. When the callback provided to `map` is a pure function, the entire operation remains predictable and side-effect free. This aligns with key functional programming concepts like immutability and referential transparency.

Listing 3.6 Using map with a pure function

```
const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2);
console.log(doubled); // [2, 4, 6]
```

For this topic, are you able to answer these follow-up/probing questions?

- "Is `console.log()` in a function considered a side effect? Why?"
- "How would you implement an immutable update to a deeply nested object in JavaScript?"
- "What is referential transparency, and why is it important for reasoning about code?"
- "Why might a language like JavaScript favor functional techniques even though it is multi-paradigm?"
- "Can a function that reads from a global constant (but does not modify it) still be considered pure? Why or why not?"

Question (applied skill): "How do you use currying and higher-order functions?"

Analysis: This question tests the ability to recognize, implement, and explain currying and higher-order functions. Candidates are expected to demonstrate how these patterns lead to cleaner, more modular, and maintainable code—skills that are essential for solving real-world JavaScript challenges.

Answer: Currying transforms a function with multiple arguments into a sequence of functions, each taking a single argument. This enables partial application, allowing you to pre-fill some arguments and create specialized functions for later use. Currying enhances code reusability and modularity, making it easier to compose complex behaviors from simple, single-responsibility functions.